

2026-05-05-p1-f01-auto-compaction

Phase 1 / Feature 1 — Auto-Compaction System (claude-code) Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers: subagent-driven-development (recommended) or superpowers: executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Land claude-code’s automatic context compaction (80% threshold trigger + thrashing detection + summary metadata) inside HelixCode by extending the existing internal/llm/compression/ infrastructure rather than building a parallel system.

Architecture: Three additions on top of the pre-existing CompressionCoordinator: (1) two new methods on the Provider interface (GetContextWindow() int, CountTokens(text string) (int, error)) so the trigger can compute window-percentage at runtime; (2) a ThrashingGuard wrapper around CompressionCoordinator.Compress() that aborts after 3 consecutive compactions with no intervening user message; (3) a CompactionMetadata field attached to assistant messages after a compaction, preserving forensic linkage between summary and original. Wire all three into internal/agent/agent.go’s message loop and internal/session/manager.go’s lifecycle. Unit tests with mocked LLM, integration test against real Anthropic provider via docker-compose, plus a Challenge with expected.json.

Tech Stack: Go 1.26, testify v1.11, the existing digital.vasic.containers Go library for docker-compose orchestration, the existing internal/llm/compression/ package, the existing internal/llm/compressioniface/ interface boundary.

Source spec: docs/superpowers/specs/2026-05-04-cli-agent-fusion-synthesis-design.md §4.1 + docs/improvements/04_main_plan_step_02/ kimi_agent_helix_cli_integration_blueprint/porting_claude_code.md §“Feature 1: Auto-Compaction System”.

Critical pre-flight finding: the porting doc was authored without awareness of internal/llm/compression/ (full coordinator + 3 strategies + config + stats already in place). This plan extends rather than duplicates. The porting doc’s internal/context/compaction.go + internal/context/token_counter.go are NOT created — those would be parallel infrastructure.

File Structure

NEW files

Path	Responsibility
helix_code/internal/llm/compression/thrashing.go	

Path	Responsibility
	ThrashingGuard wrapper: tracks consecutive-compaction count, aborts on 3-without-user-message
helix_code/internal/llm/compression/thrashing_test.go	Unit tests for ThrashingGuard (mocks allowed per CONST-002)
helix_code/internal/llm/compression/metadata.go	CompactionMetadata + helpers to attach/read it from *compressioniface.Message
helix_code/internal/llm/compression/metadata_test.go	Unit tests for metadata
helix_code/internal/llm/compression/auto_compact.go	AutoCompactor — uses Provider's GetContextWindow()/CountTokens() to gate triggering at 80% threshold; calls CompressionCoordinator.Compress() via ThrashingGuard; attaches metadata
helix_code/internal/llm/compression/auto_compact_test.go	Unit tests with mocked Provider + mocked CompressionCoordinator
helix_code/tests/integration/auto_compaction_integration_test.go	- tags=integration, no mocks, real Anthropic provider, real PostgreSQL session store
helix_code/tests/e2e/challenges/auto-compaction/run.sh	End-to-end Challenge invoking the agent against a long-conversation fixture
helix_code/tests/e2e/challenges/auto-compaction/expected.json	Runtime-evidence assertions

MODIFIED files

Path	What changes
helix_code/internal/llm/missing_types.go	Add GetContextWindow() int and CountTokens(text string) (int, error) to Provider interface
helix_code/internal/llm/anthropic_provider.go	Implement both new methods (uses tiktoken if available; claude-3-5-sonnet's 200k context)
helix_code/internal/llm/azure_provider.go	Implement both new methods (delegate to underlying OpenAI Azure tokenizer)
helix_code/internal/llm/bedrock_provider.go	Implement both new methods
helix_code/internal/llm/copilot_provider.go	Implement both new methods
helix_code/internal/llm/<every other *_provider.go>	Implement both new methods — minimal: char-based fallback (1 token ≈ 4 chars), 200k window default. Per-provider real tokenizers are deferred to a Phase 3 sub-spec.

Path	What changes
helix_code/internal/agent/agent.go	Insert <code>AutoCompactor.MaybeCompact()</code> call before each LLM request in the message loop
helix_code/internal/session/manager.go	Reset <code>ThrashingGuard.consecutiveCount</code> when a user message is added to the session
helix_code/docs/improvements/05_phase_0_evidence.md	(NOT touched — that's Phase 0's evidence file. Phase 1 evidence goes elsewhere.)
docs/improvements/06_phase_1_evidence.md	NEW — accumulated Phase 1 evidence; first section is Feature 1
docs/improvements/PROGRESS.md	Move Phase 1 status from pending → active; add Feature 1 task list

Cross-cutting conventions

- **Branch:** stay on `main`. Phase 0's authorisation extends through the programme.
- **Commit format (per spec §7.2):**

```
<type>(P1-F01-<task-id>): <subject>
```

```
<short description>
```

```
Phase: P1
```

```
Feature: F01 — Auto-Compaction
```

```
Task: P1-F01-<task-id>
```

```
Evidence: <pasted runtime output OR pointer>
```

```
Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
```
- **Push:** at the end of every task; all four remotes (`github`, `gitlab`, `origin`, `upstream`); verify convergence with `git ls-remote --heads <r> main`.
- **Working directory:** task steps assume `cd /run/media/milosvasic/DATA4TB/Projects/HelixCode` unless otherwise stated.
- `make verify-foundation` is currently `exit=2` (LLMsVerifier divergence carry-forward). Per Phase 0 close-out, this is an accepted carry-forward; Feature 1 work proceeds with that gate marked WARN.
- **No third-party submodule modifications.** No commits inside any submodule.
- **CONST-042 absolute** (no secret leakage). **CONST-043 absolute** (no force-push).

Task 1: Bootstrap Phase 1 evidence file + advance PROGRESS.md

Files: - Create: docs/improvements/06_phase_1_evidence.md - Modify: docs/improvements/PROGRESS.md



Step 1.1: Create the Phase 1 evidence header

Phase 1 Evidence Log – claude-code feature porting

Each feature's acceptance check output is pasted below with a timestamp.

This file is the rolled-up forensic record per Article XI §11.9.

Spec: `docs/superpowers/specs/2026-05-04-cli-agent-fusion-synthesis-design.md` §4.1

Plan (Feature 1): `docs/superpowers/plans/2026-05-05-p1-f01-auto-compaction.md`



Step 1.2: Update PROGRESS.md

Open docs/improvements/PROGRESS.md. Move Phase 1 row in “Phase status” table from pending to active with Started: 2026-05-05. Add a new “Active feature” sub-section under “Current focus” naming F01 – Auto-Compaction. Add new active task list:

Active feature task list (P1-F01: Auto-Compaction)

- [-] P1-F01-T01 – bootstrap Phase 1 evidence + advance PROGRESS
- [] P1-F01-T02 – add GetContextWindow + CountTokens to Provider interface
- [] P1-F01-T03 – implement Provider methods across all *_provider.go
- [] P1-F01-T04 – ThrashingGuard with TDD
- [] P1-F01-T05 – CompactionMetadata with TDD
- [] P1-F01-T06 – AutoCompactor with TDD
- [] P1-F01-T07 – wire AutoCompactor into internal/agent/agent.go
- [] P1-F01-T08 – wire ThrashingGuard reset into internal/session/manager.go
- [] P1-F01-T09 – integration test against real Anthropic provider
- [] P1-F01-T10 – Challenge with expected.json + runtime evidence
- [] P1-F01-T11 – Feature 1 close-out + push



Step 1.3: Commit + push

```

git add docs/improvements/06_phase_1_evidence.md docs/
      improvements/PROGRESS.md
git commit -m "$(cat <<'EOF'
docs(P1-F01-T01): bootstrap Phase 1 evidence file; advance to F01
      active

Phase: P1
Feature: F01 — Auto-Compaction
Task:    P1-F01-T01
Evidence: file written; cat verified.

Co-Authored-By: Claude Opus 4.7 (1M context)
      <noreply@anthropic.com>
EOF
)"
for r in github gitlab origin upstream; do git push $r main; done

```

Task 2: Add GetContextWindow() + CountTokens() to Provider interface

Files: - Modify: `helix_code/internal/llm/missing_types.go` (or wherever type Provider interface lives)



Step 2.1: Locate and read the Provider interface

```

grep -nE "^type Provider interface" helix_code/internal/llm/
      missing_types.go

```

Use Read tool to confirm the interface body and find the closing }.



Step 2.2: Add the two new methods

Use the Edit tool to add inside the Provider interface body, between `Close()` error and the closing brace, the following:

```

// GetContextWindow returns the maximum number of tokens the
      active model
// can hold in a single context window. Used by the auto-
      compaction system
// to compute the 80%-trigger threshold.
GetContextWindow() int

// CountTokens returns an estimated token count for the given
      text.
// Implementations SHOULD use the provider's native tokenizer
      when available

```

```
// (e.g., tiktoken for OpenAI, anthropic-tokenizer for Anthropic)
// and MUST
// fall back to a conservative char-based estimate (1 token ≈ 4
// chars) when
// no native tokenizer is reachable. Returns 0 + nil for empty
// text.
CountTokens(text string) (int, error)
```



Step 2.3: Verify the file compiles in isolation

```
cd HelixCode && go build ./internal/llm/missing_types.go 2>&1 |
head -10
```

Expected: file does NOT compile in isolation if other files in the package implement Provider — but `go build ./internal/llm/...` will fail because every concrete Provider implementation now lacks two methods. That's intended; Task 3 fixes them.



Step 2.4: Commit (interface change only — package will not build until T03)

```
git add helix_code/internal/llm/missing_types.go
git commit -m "$(cat <<'EOF'
feat(P1-F01-T02): add GetContextWindow + CountTokens to Provider
interface
```

```
Foundation for claude-code-style auto-compaction (80%-window
trigger).
```

```
Concrete provider implementations are added in T03; the package
WILL NOT
```

```
BUILD between this commit and the next. This is intentional —
TDD's
```

```
"write the failing test, make it pass" approach at the interface
level.
```

Phase: P1

Feature: F01 — Auto-Compaction

Task: P1-F01-T02

Co-Authored-By: Claude Opus 4.7 (1M context)

<noreply@anthropic.com>

EOF

)"

```
for r in github gitlab origin upstream; do git push $r main; done
```

Task 3: Implement GetContextWindow() + CountTokens() on every existing Provider

Files: - Modify: every `helix_code/internal/llm/*_provider.go` (anthropic, azure, bedrock, copilot, openai, ollama, google, gemini, deepseek, groq, mistral, xai, openrouter, llama_cpp, ...)



Step 3.1: Enumerate every existing Provider implementation

```
cd HelixCode && grep -lE "func \(. * \*[A-Za-z]+Provider\)
    GetType\(\)" internal/llm/*_provider.go
```

Each file in the list needs the two new methods added.



Step 3.2: Define a shared char-based fallback helper

Create `helix_code/internal/llm/token_fallback.go`:

```
package llm

import "math"

// CharBasedTokenCount returns a conservative token estimate (1
// token ≈ 3.5 chars).
// Used as a fallback by providers that don't have a native
// tokenizer reachable
// at runtime. Per CONST-035, individual providers SHOULD upgrade
// to their
// native tokenizer (Phase 3 sub-spec); this fallback keeps the
// Provider
// interface contract honoured in the meantime.
func CharBasedTokenCount(text string) (int, error) {
    if text == "" {
        return 0, nil
    }
    return int(math.Ceil(float64(len(text)) / 3.5)), nil
}
```



Step 3.3: Add fallback methods to every Provider implementation

For each `*_provider.go`, append:

```
// GetContextWindow returns the model's context window size in
// tokens.
// Default: 200_000 (Claude 3.5 Sonnet's window) — providers MUST
// override
// with their model's actual window where it differs.
```

```

func (p *<TypeName>) GetContextWindow() int {
    return 200_000
}

// CountTokens returns an estimated token count for text.
// Default: char-based fallback (1 token ≈ 3.5 chars) – providers
// SHOULD
// override with their native tokenizer (Phase 3 sub-spec).
func (p *<TypeName>) CountTokens(text string) (int, error) {
    return CharBasedTokenCount(text)
}

```

Replace <TypeName> per file (AnthropicProvider, AzureProvider, BedrockProvider, CopilotProvider, etc.).

For Anthropic specifically, override GetContextWindow to return the actual window for the active model (200_000 for Claude 3.5 Sonnet, 100_000 for Claude 2, etc.) — read the model name from p.config.Model (or wherever it’s stored) and dispatch:

```

func (p *AnthropicProvider) GetContextWindow() int {
    switch {
    case strings.Contains(p.modelName, "claude-3-5-sonnet"):
        return 200_000
    case strings.Contains(p.modelName, "claude-3-opus"):
        return 200_000
    case strings.Contains(p.modelName, "claude-3-haiku"):
        return 200_000
    case strings.Contains(p.modelName, "claude-2"):
        return 100_000
    default:
        return 200_000
    }
}

```



Step 3.4: Compile

```
cd HelixCode && go build ./internal/llm/...
```

Expected: exit 0 with no errors. If any provider was missed, the compiler will say “does not implement Provider”.



Step 3.5: Smoke-test

```

cd HelixCode && go test -v -run
    TestAnthropicProvider_GetContextWindow ./internal/llm/
    anthropic_provider_test.go ./internal/llm/
    anthropic_provider.go 2>&1 | tail -10

```


If test doesn't exist, that's fine — Task 4+ adds tests. Just confirm the build is clean.



Step 3.6: Commit + push

```
git add helix_code/internal/llm/token_fallback.go helix_code/
      internal/llm/*_provider.go
git commit -m "$(cat <<'EOF'
feat(P1-F01-T03): implement GetContextWindow + CountTokens on
      every Provider
```

Adds shared char-based fallback (CharBasedTokenCount, 1 token ≈ 3.5 chars) plus per-provider methods. Anthropic provider gets a model-aware GetContextWindow that dispatches on claude-3-{5-sonnet,opus,haiku} and claude-2; other providers default to 200k pending Phase 3 native-tokenizer upgrade.

```
go build ./internal/llm/... is now clean.
```

Phase: P1

Feature: F01 — Auto-Compaction

Task: P1-F01-T03

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF

)"

```
for r in github gitlab origin upstream; do git push $r main; done
```

Task 4: ThrashingGuard with TDD

Files: - Create: helix_code/internal/llm/compression/thrashing.go - Create: helix_code/internal/llm/compression/thrashing_test.go



Step 4.1: Write the failing test

```
// helix_code/internal/llm/compression/thrashing_test.go
package compression

import (
    "errors"
    "testing"

    "github.com/stretchr/testify/assert"
```

```

    "github.com/stretchr/testify/require"
)

func TestThrashingGuard_AllowsFirstThreeCompactions(t
    *testing.T) {
    g := NewThrashingGuard(3)
    for i := 0; i < 3; i++ {
        err := g.RecordCompaction()
        require.NoError(t, err, "compaction %d should not
            error", i+1)
    }
}

func TestThrashingGuard_AbortsFourthConsecutive(t *testing.T) {
    g := NewThrashingGuard(3)
    for i := 0; i < 3; i++ {
        _ = g.RecordCompaction()
    }
    err := g.RecordCompaction()
    require.Error(t, err)
    assert.True(t, errors.Is(err, ErrThrashing), "fourth
        consecutive must be ErrThrashing")
}

func TestThrashingGuard_ResetOnUserMessage(t *testing.T) {
    g := NewThrashingGuard(3)
    for i := 0; i < 3; i++ {
        _ = g.RecordCompaction()
    }
    g.NoteUserMessage()
    err := g.RecordCompaction()
    require.NoError(t, err, "after NoteUserMessage, the guard
        should reset")
}

func TestThrashingGuard_ZeroLimitAllowsNothing(t *testing.T) {
    g := NewThrashingGuard(0)
    err := g.RecordCompaction()
    require.Error(t, err)
}

```



Step 4.2: Run test to verify it fails

```

cd HelixCode && go test -v -run TestThrashingGuard ./internal/
    llm/compression/...

```

Expected: FAIL with undefined: NewThrashingGuard etc.



Step 4.3: Write the minimal implementation

```
// helix_code/internal/llm/compression/thrashing.go
package compression

import (
    "errors"
    "sync"
)

// ErrThrashing is returned when the guard detects N consecutive
// compactions
// with no intervening user message. Per claude-code's auto-
// compaction
// design, this signals that the agent is "overwhelmed" and the
// caller
// should surface the error to the human rather than silently
// looping.
var ErrThrashing = errors.New("compaction thrashing: N
    consecutive compactions with no user message")

// ThrashingGuard tracks consecutive compactions and aborts when
// a configured
// threshold is reached without an intervening user message.
type ThrashingGuard struct {
    threshold int
    mu        sync.Mutex
    count     int
}

// NewThrashingGuard returns a guard configured with the given
// threshold.
// claude-code uses threshold=3.
func NewThrashingGuard(threshold int) *ThrashingGuard {
    return &ThrashingGuard{threshold: threshold}
}

// RecordCompaction increments the consecutive-compaction counter
// and
// returns ErrThrashing if the counter has reached the threshold.
// The increment happens before the comparison, so a threshold of
// 3
// allows compactions 1, 2, 3 and rejects compaction 4.
func (g *ThrashingGuard) RecordCompaction() error {
    g.mu.Lock()
    defer g.mu.Unlock()
    g.count++
    if g.count > g.threshold {
```

```

        return ErrThrashing
    }
    return nil
}

// NoteUserMessage resets the consecutive-compaction counter to
// zero.
// The session manager must call this whenever a user message is
// appended
// to the conversation.
func (g *ThrashingGuard) NoteUserMessage() {
    g.mu.Lock()
    defer g.mu.Unlock()
    g.count = 0
}

```



Step 4.4: Run test to verify it passes

```
cd HelixCode && go test -v -run TestThrashingGuard ./internal/
llm/compression/...
```

Expected: 4/4 PASS.



Step 4.5: Commit + push

```
git add helix_code/internal/llm/compression/thrashing.go
      helix_code/internal/llm/compression/thrashing_test.go
git commit -m "$(cat <<'EOF'
feat(P1-F01-T04): add ThrashingGuard with TDD

```

Detects N consecutive compactions with no intervening user message;
returns ErrThrashing when threshold exceeded. Per claude-code's auto-compaction design (threshold=3). 4/4 unit tests pass.

Phase: P1
Feature: F01 – Auto-Compaction
Task: P1-F01-T04

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF

)"

```
for r in github gitlab origin upstream; do git push $r main; done
```

Task 5: CompactionMetadata with TDD

Files: - Create: `helix_code/internal/llm/compression/metadata.go` - Create: `helix_code/internal/llm/compression/metadata_test.go`



Step 5.1: Write the failing test

```
// helix_code/internal/llm/compression/metadata_test.go
package compression

import (
    "testing"
    "time"

    "github.com/stretchr/testify/require"
    "dev.helix.code/internal/llm/compressioniface"
)

func TestCompactionMetadata_RoundTrip(t *testing.T) {
    original := &CompactionMetadata{
        OriginalMessageCount: 50,
        OriginalTokenCount:   45_000,
        CompactedTokenCount:  5_000,
        SummarizedAt:         time.Date(2026, 5, 5, 10, 0, 0, 0,
            time.UTC),
        SummaryText:
            "User asked about X; agent answered Y; key decision: Z.",
        TopicsCovered:      []string{"X", "Y", "Z"},
        KeyDecisions:         []string{"chose Z over W"},
    }
    msg := &compressioniface.Message{Role: "assistant", Content: "..."}
    require.NoError(t, AttachCompactionMetadata(msg, original))
    round, ok := ReadCompactionMetadata(msg)
    require.True(t, ok)
    require.Equal(t, original.OriginalMessageCount,
        round.OriginalMessageCount)
    require.Equal(t, original.SummaryText, round.SummaryText)
    require.Equal(t, original.TopicsCovered, round.TopicsCovered)
}

func TestReadCompactionMetadata_Absent(t *testing.T) {
    msg := &compressioniface.Message{Role: "assistant", Content:
        "ordinary message"}
    _, ok := ReadCompactionMetadata(msg)
    require.False(t, ok)
}
```

```
func TestAttachCompactionMetadata_RejectsNonAssistantRole(t
    *testing.T) {
    msg := &compressioniface.Message{Role: "user", Content:
        "..."}
    err := AttachCompactionMetadata(msg, &CompactionMetadata{})
    require.Error(t, err, "metadata is for assistant messages
        only")
}
```



Step 5.2: Run test (must fail)

```
cd HelixCode && go test -v -run TestCompactionMetadata -run
    TestReadCompactionMetadata -run
    TestAttachCompactionMetadata ./internal/llm/
    compression/...
```

Expected: undefined identifiers.



Step 5.3: Implement

```
// helix_code/internal/llm/compression/metadata.go
package compression

import (
    "encoding/json"
    "errors"
    "time"

    "dev.helix.code/internal/llm/compressioniface"
)

// MetadataKey is the key under compressioniface.Message.Metadata
// where the
// CompactionMetadata blob is stored (JSON-encoded). Stable
// across versions
// to allow forensic tooling to find compaction summaries.
const MetadataKey = "context_management"

// ErrInvalidMessageRole is returned when
// AttachCompactionMetadata is given
// a message whose role is not "assistant".
var ErrInvalidMessageRole = errors.New("compaction metadata is
    for assistant messages only")

// CompactionMetadata captures the forensic record of a single
// compaction
```

```

// event. It is attached to the assistant message that REPLACES
// the
// summarized portion of the conversation history.
type CompactionMetadata struct {
    OriginalMessageCount int
        `json:"original_message_count"`
    OriginalTokenCount   int        `json:"original_token_count"`
    CompactedTokenCount  int        `json:"compacted_token_count"`
    SummarizedAt         time.Time `json:"summarized_at"`
    SummaryText          string    `json:"summary_text"`
    TopicsCovered        []string
        `json:"topics_covered,omitempty"`
    KeyDecisions         []string
        `json:"key_decisions,omitempty"`
}

// AttachCompactionMetadata serialises the metadata as JSON and
// stores it
// under MetadataKey in the message's Metadata map. Creates the
// map if nil.
func AttachCompactionMetadata(msg *compressioniface.Message, m
    *CompactionMetadata) error {
    if msg == nil {
        return errors.New("nil message")
    }
    if msg.Role != "assistant" {
        return ErrInvalidMessageRole
    }
    if m == nil {
        return errors.New("nil metadata")
    }
    if msg.Metadata == nil {
        msg.Metadata = make(map[string]interface{})
    }
    encoded, err := json.Marshal(m)
    if err != nil {
        return err
    }
    msg.Metadata[MetadataKey] = string(encoded)
    return nil
}

// ReadCompactionMetadata returns the CompactionMetadata stored
// on the
// message, or (nil, false) if absent or unparseable.
func ReadCompactionMetadata(msg *compressioniface.Message)
    (*CompactionMetadata, bool) {
    if msg == nil || msg.Metadata == nil {

```

```

        return nil, false
    }
    raw, ok := msg.Metadata[MetadataKey]
    if !ok {
        return nil, false
    }
    str, ok := raw.(string)
    if !ok {
        return nil, false
    }
    var m CompactionMetadata
    if err := json.Unmarshal([]byte(str), &m); err != nil {
        return nil, false
    }
    return &m, true
}

```



Step 5.4: Verify the existing `compressioniface.Message` has a `Metadata` field

```

grep -n "Metadata" helix_code/internal/llm/compressioniface/
interface.go | head -10

```

Expected: line `Metadata map[string]interface{}` present in the `Conversation` and/or `Message` struct. If the field is on `Conversation` rather than `Message`, ADJUST the `metadata.go` above to attach to `Conversation.Metadata` instead.



Step 5.5: Run tests

```

cd HelixCode && go test -v -run TestCompactionMetadata -run
    TestReadCompactionMetadata -run
    TestAttachCompactionMetadata ./internal/llm/
    compression/...

```

Expected: 3/3 PASS.



Step 5.6: Commit + push

```

git add helix_code/internal/llm/compression/metadata.go
    helix_code/internal/llm/compression/metadata_test.go
git commit -m "$(cat <<'EOF'
feat(P1-F01-T05): add CompactionMetadata with TDD

```

```

Attach/read forensic metadata on assistant messages: original/
    compacted
counts, summary text, topics covered, key decisions. Stored under
MetadataKey="context_management" as JSON in Message.Metadata. 3/3
unit

```


tests pass.

Phase: P1

Feature: F01 — Auto-Compaction

Task: P1-F01-T05

Co-Authored-By: Claude Opus 4.7 (1M context)

<noreply@anthropic.com>

EOF

)"

for r in github gitlab origin upstream; do git push \$r main; done

Task 6: AutoCompactor — the 80%-trigger orchestrator

Files: - Create: helix_code/internal/llm/compression/auto_compact.go -

Create: helix_code/internal/llm/compression/auto_compact_test.go



Step 6.1: Write the failing test

```
// helix_code/internal/llm/compression/auto_compact_test.go
package compression

import (
    "context"
    "testing"

    "github.com/stretchr/testify/mock"
    "github.com/stretchr/testify/require"
    "dev.helix.code/internal/llm/compressioniface"
)

// mockProvider implements just GetContextWindow + CountTokens
// for AutoCompactor.
type mockProvider struct {
    mock.Mock
}

func (m *mockProvider) GetContextWindow() int {
    return m.Called().Int(0)
}

func (m *mockProvider) CountTokens(text string) (int, error) {
    args := m.Called(text)
    return args.Int(0), args.Error(1)
}
```

```

// mockCoordinator stub of
    compressioniface.CompressionCoordinator
type mockCoordinator struct {
    mock.Mock
}

func (m *mockCoordinator) Compress(ctx context.Context, conv
    *compressioniface.Conversation)
    (*compressioniface.CompressionResult, error) {
    args := m.Called(ctx, conv)
    if args.Get(0) == nil {
        return nil, args.Error(1)
    }
    return args.Get(0).(*compressioniface.CompressionResult),
        args.Error(1)
}
func (m *mockCoordinator) ShouldCompress(conv
    *compressioniface.Conversation) (bool, string) {
    return false, ""
}
func (m *mockCoordinator)
    EstimateCompression(*compressioniface.Conversation)
    (*compressioniface.CompressionEstimate, error) { return
    nil, nil }
func (m *mockCoordinator) GetStats()
    *compressioniface.CompressionStats
    { return nil }
func (m *mockCoordinator) GetConfig()
    *compressioniface.Config
    { return nil }
func (m *mockCoordinator)
    UpdateConfig(*compressioniface.Config)
    {}

func TestAutoCompactor_BelowThreshold_NoCompaction(t *testing.T)
{
    prov := new(mockProvider)
    coord := new(mockCoordinator)
    ac := NewAutoCompactor(prov, coord, NewThrashingGuard(3),
        0.80)

    prov.On("GetContextWindow").Return(100_000)
    prov.On("CountTokens", mock.Anything).Return(50_000, nil) //
        50% - below 80% threshold

    conv := &compressioniface.Conversation{Messages:
        []*compressioniface.Message{{Content: "x"}}}
    result, err := ac.MaybeCompact(context.Background(), conv)
    require.NoError(t, err)
}

```

```

        require.False(t, result.WasCompacted)
    }

func TestAutoCompactor_AboveThreshold_TriggersCompression(t
    *testing.T) {
    prov := new(mockProvider)
    coord := new(mockCoordinator)
    ac := NewAutoCompactor(prov, coord, NewThrashingGuard(3),
        0.80)

    prov.On("GetContextWindow").Return(100_000)
    prov.On("CountTokens", mock.Anything).Return(85_000, nil) //
        85% – above 80% threshold
    coord.On("Compress", mock.Anything,
        mock.Anything).Return(&compressioniface.CompressionResult{},
        nil)

    conv := &compressioniface.Conversation{Messages:
        []*compressioniface.Message{{Role: "assistant", Content:
            "x"}}}}
    result, err := ac.MaybeCompact(context.Background(), conv)
    require.NoError(t, err)
    require.True(t, result.WasCompacted)
    coord.AssertCalled(t, "Compress", mock.Anything,
        mock.Anything)
}

func TestAutoCompactor_ThrashingAfterFourth(t *testing.T) {
    prov := new(mockProvider)
    coord := new(mockCoordinator)
    guard := NewThrashingGuard(3)
    ac := NewAutoCompactor(prov, coord, guard, 0.80)

    prov.On("GetContextWindow").Return(100_000)
    prov.On("CountTokens", mock.Anything).Return(85_000, nil)
    coord.On("Compress", mock.Anything,
        mock.Anything).Return(&compressioniface.CompressionResult{},
        nil)

    conv := &compressioniface.Conversation{Messages:
        []*compressioniface.Message{{Role: "assistant", Content:
            "x"}}}}
    for i := 0; i < 3; i++ {
        _, err := ac.MaybeCompact(context.Background(), conv)
        require.NoError(t, err)
    }
    _, err := ac.MaybeCompact(context.Background(), conv)

```

```
    require.ErrorIs(t, err, ErrThrashing)
}
```



Step 6.2: Run (fails)

```
cd HelixCode && go test -v -run TestAutoCompactor ./internal/llm/
compression/...
```



Step 6.3: Implement

```
// helix_code/internal/llm/compression/auto_compact.go
package compression

import (
    "context"
    "fmt"

    "dev.helix.code/internal/llm/compressioniface"
)

// TokenCounter is the subset of llm.Provider that AutoCompactor
// needs.
// Defined here to keep the dependency small + mock-friendly.
type TokenCounter interface {
    GetContextWindow() int
    CountTokens(text string) (int, error)
}

// AutoCompactionResult is the outcome of a MaybeCompact call.
type AutoCompactionResult struct {
    WasCompacted    bool
    TokensBefore    int
    TokensAfter     int
    WindowSize      int
    ThresholdRatio  float64
}

// AutoCompactor wraps an existing CompressionCoordinator with
// claude-code's
// 80%-window trigger + thrashing-detection semantics.
type AutoCompactor struct {
    tokens          TokenCounter
    coord           compressioniface.CompressionCoordinator
    guard           *ThrashingGuard
    thresholdRatio  float64
}
```

```

// NewAutoCompactor returns an AutoCompactor configured with the
// given
// token-counter, compression coordinator, thrashing guard, and
// threshold
// ratio (e.g., 0.80 for claude-code's 80% trigger).
func NewAutoCompactor(tokens TokenCounter, coord
    compressioniface.CompressionCoordinator, guard
    *ThrashingGuard, thresholdRatio float64) *AutoCompactor {
    return &AutoCompactor{tokens: tokens, coord: coord, guard:
        guard, thresholdRatio: thresholdRatio}
}

// MaybeCompact checks whether the conversation has crossed the
// threshold
// and, if so, runs compression via the coordinator (gated by the
// thrashing guard). Returns ErrThrashing if the guard rejects
// the call.
func (a *AutoCompactor) MaybeCompact(ctx context.Context, conv
    *compressioniface.Conversation) (*AutoCompactionResult,
    error) {
    if conv == nil || len(conv.Messages) == 0 {
        return &AutoCompactionResult{}, nil
    }

    // Sum tokens across all messages
    total := 0
    for _, m := range conv.Messages {
        n, err := a.tokens.CountTokens(m.Content)
        if err != nil {
            return nil, fmt.Errorf("counting tokens: %w", err)
        }
        total += n
    }
    window := a.tokens.GetContextWindow()
    threshold := int(float64(window) * a.thresholdRatio)

    result := &AutoCompactionResult{
        TokensBefore: total,
        WindowSize:    window,
        ThresholdRatio: a.thresholdRatio,
    }

    if total < threshold {
        return result, nil
    }

    if err := a.guard.RecordCompaction(); err != nil {
        return result, err
    }

```

```

    }

    cr, err := a.coord.Compress(ctx, conv)
    if err != nil {
        return result, fmt.Errorf("compress: %w", err)
    }

    result.WasCompacted = true
    if cr != nil {
        result.TokensAfter = cr.TokensAfter
    }
    return result, nil
}

```



Step 6.4: Verify `compressioniface.CompressionResult` **has** `TokensAfter`

```

grep -n "TokensAfter" helix_code/internal/llm/compressioniface/
    *.go

```

If absent, adjust the assignment to whichever field the existing struct exposes (e.g., `cr.CompressedTokens`).



Step 6.5: Run tests

```

cd HelixCode && go test -v -run TestAutoCompactor ./internal/llm/
    compression/...

```

Expected: 3/3 PASS.



Step 6.6: Commit + push

```

git add helix_code/internal/llm/compression/auto_compact.go
    helix_code/internal/llm/compression/auto_compact_test.go
git commit -m "$(cat <<'EOF'
feat(P1-F01-T06): add AutoCompactor 80%-trigger orchestrator

```

```

Wraps existing CompressionCoordinator with claude-code's auto-
    compaction
semantics: configurable threshold (default 80%), thrashing guard,
forensic result struct. 3/3 unit tests pass with mocked Provider
    +
CompressionCoordinator.

```

```

Phase: P1
Feature: F01 — Auto-Compaction
Task:    P1-F01-T06

```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF

)"

for r in github gitlab origin upstream; do git push \$r main; done

Task 7: Wire AutoCompactor into internal/agent/agent.go

Files: - Modify: helix_code/internal/agent/agent.go



Step 7.1: Read agent.go to find the message-loop hot path

```
grep -nE "Generate|GenerateStream|sendToProvider|providerCall|llmRequest" helix_code/internal/agent/agent.go | head -10
```

Identify the exact line where the agent calls the provider's `Generate()` (or wherever the LLM round-trip happens). The auto-compactor must run BEFORE that call.



Step 7.2: Add AutoCompactor field + constructor wiring

Use Edit tool:

1. Add an `AutoCompactor *compression.AutoCompactor` field to the `Agent` struct.
2. In `NewAgent` (or equivalent constructor), accept an optional `autoCompactor` parameter (use functional-option pattern if codebase uses one; otherwise add as a constructor arg with documentation).
3. Default behaviour when `autoCompactor` is `nil`: skip compaction (graceful degradation; not all callers need it during transition).



Step 7.3: Insert MaybeCompact call before Generate

Just before the line where `provider.Generate(ctx, req)` is called (or `provider.GenerateStream`), insert:

```
if a.autoCompactor != nil {
    conv := buildConversationFromHistory(a.history)
    if _, err := a.autoCompactor.MaybeCompact(ctx, conv); err !=
        nil {
        return nil, fmt.Errorf("auto-compaction: %w", err)
    }
}
```

If `buildConversationFromHistory` doesn't exist, add a small helper function in `internal/agent/agent.go` that converts the agent's internal message history into `*compressioniface.Conversation`.



Step 7.4: Compile

```
cd HelixCode && go build ./internal/agent/...
```

Expected: exit 0.



Step 7.5: Run existing agent tests to ensure no regression

```
cd HelixCode && go test -v ./internal/agent/... 2>&1 | tail -30
```

Expected: PASS rate identical to pre-task baseline. If tests now require an `autoCompactor` argument, update them to pass `nil` (graceful degradation).



Step 7.6: Commit + push

```
git add helix_code/internal/agent/agent.go
git commit -m "$(cat <<'EOF'
feat(P1-F01-T07): wire AutoCompactor into agent.go message loop

Agent struct gains an AutoCompactor field; MaybeCompact() runs
before
each LLM Generate call. Nil autoCompactor = graceful degradation
(no-op).

Phase: P1
Feature: F01 – Auto-Compaction
Task:    P1-F01-T07

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF
)"
for r in github gitlab origin upstream; do git push $r main; done
```

Task 8: Wire `ThrashingGuard.NoteUserMessage()` into session manager

Files: - Modify: `helix_code/internal/session/manager.go`



Step 8.1: Locate the user-message-append path


```
grep -nE "AppendMessage|AddUserMessage|UserMessage|
        user.*[Mm]essage" helix_code/internal/session/manager.go
| head -10
```

Identify where a user-role message is added to the session's message list. This is where ThrashingGuard.NoteUserMessage() must be called.



Step 8.2: Add ThrashingGuard reference to Manager struct

```
type Manager struct {
    // ... existing fields ...
    thrashingGuard *compression.ThrashingGuard // optional; nil
    = no auto-compaction wiring
}
```

Constructor accepts the guard via a setter or option:

```
func (m *Manager) SetThrashingGuard(g
    *compression.ThrashingGuard) {
    m.mu.Lock()
    defer m.mu.Unlock()
    m.thrashingGuard = g
}
```



Step 8.3: Call NoteUserMessage on user-message append

In the function that appends a user message, add (after the message is successfully appended):

```
if m.thrashingGuard != nil {
    m.thrashingGuard.NoteUserMessage()
}
```



Step 8.4: Compile + test

```
cd HelixCode && go build ./internal/session/...
cd HelixCode && go test -v ./internal/session/... 2>&1 | tail -10
```



Step 8.5: Commit + push

```
git add helix_code/internal/session/manager.go
git commit -m "$(cat <<'EOF'
feat(P1-F01-T08): wire ThrashingGuard.NoteUserMessage into
    session manager"
```

When a user message is appended to a session, the manager resets the

auto-compactor's thrashing guard. Optional wiring (nil guard = no-op).

Phase: P1

Feature: F01 — Auto-Compaction

Task: P1-F01-T08

Co-Authored-By: Claude Opus 4.7 (1M context)

<noreply@anthropic.com>

EOF

)"

for r in github gitlab origin upstream; do git push \$r main; done

Task 9: Integration test — real Anthropic provider, no mocks

Files: - Create: helix_code/tests/integration/
auto_compaction_integration_test.go



Step 9.1: Verify make test-infra-up brings up the docker-compose stack

```
cd HelixCode && make test-infra-up 2>&1 | tail -10
```

If this fails, document and skip the test (it cannot run without infra). This is acceptable as a CI-level artifact only if the SKIP carries SKIP-OK: #<ticket>.



Step 9.2: Write the integration test

```
//go:build integration
```

```
// +build integration
```

```
// helix_code/tests/integration/
```

```
    auto_compaction_integration_test.go
```

```
package integration
```

```
import (
```

```
    "context"
```

```
    "os"
```

```
    "strings"
```

```
    "testing"
```

```
    "time"
```

```
    "github.com/stretchr/testify/require"
```

```
    "dev.helix.code/internal/llm"
```

```
    "dev.helix.code/internal/llm/compression"
```

```
    "dev.helix.code/internal/llm/compressioniface"
```

```
)
```

```

func TestAutoCompaction_IntegrationLargeConversation(t
    *testing.T) {
    apiKey := os.Getenv("HELIX_LLM_ANTHROPIC_KEY")
    if apiKey == "" {
        t.Skip("SKIP-OK: #P1-F01-INT - HELIX_LLM_ANTHROPIC_KEY
            not set; integration test requires real Anthropic
            credentials")
    }

    // Set up real Anthropic provider
    prov, err := llm.NewAnthropicProvider(&llm.AnthropicConfig{
        APIKey: apiKey,
        Model:  "claude-3-5-sonnet-20241022",
    })
    require.NoError(t, err)
    defer prov.Close()

    // Build a deliberately-too-long conversation: 50 messages
    // averaging 5000 chars each
    // = ~250,000 chars ≈ 71,000 tokens, well above 80% of 200k
    // window.
    conv := &compressioniface.Conversation{
        ID:          "p1-f01-int-test",
        CreatedAt:   time.Now(),
        Messages:    make([]*compressioniface.Message, 0, 50),
    }
    for i := 0; i < 50; i++ {
        role := "user"
        if i%2 == 1 {
            role = "assistant"
        }
        conv.Messages = append(conv.Messages,
            &compressioniface.Message{
                Role:      compressioniface.MessageRole(role),
                Content:  strings.Repeat("This is filler content to
                    consume tokens. ", 100),
                ID:      "msg-" + string(rune(i)),
            })
    }

    // Set up the existing CompressionCoordinator with semantic-
    // summarization strategy
    coord := compression.NewCompressionCoordinator(
        prov,
        compression.WithStrategy(compression.StrategySemanticSummarization)
    )
    guard := compression.NewThrashingGuard(3)

```

```

ac := compression.NewAutoCompactor(prov, coord, guard, 0.80)

// First call SHOULD trigger compaction
ctx, cancel := context.WithTimeout(context.Background(),
    60*time.Second)
defer cancel()
result, err := ac.MaybeCompact(ctx, conv)
require.NoError(t, err)
require.True(t, result.WasCompacted, "conversation at 71k
    tokens should trigger compaction at 80% of 200k window")
require.Less(t, result.TokensAfter, result.TokensBefore,
    "post-compaction tokens must be less than pre-compaction
    tokens")

// Verify the compacted conversation has fewer messages or
// has metadata attached
hasMetadata := false
for _, m := range conv.Messages {
    if _, ok := compression.ReadCompactionMetadata(m); ok {
        hasMetadata = true
        break
    }
}
// Either fewer messages OR explicit metadata is acceptable;
// both is preferred.
require.True(t, hasMetadata || len(conv.Messages) < 50,
    "after compaction, conversation must have either
    CompactionMetadata or fewer messages")
}

```



Step 9.3: Run the test (requires real key)

```

cd HelixCode && go test -v -tags=integration -run
    TestAutoCompaction_Integration ./tests/integration/...
    -timeout 90s 2>&1 | tail -30

```

Expected: PASS if HELIX_LLM_ANTHROPIC_KEY is set; SKIP-OK if not. Either way, the test is committed and runs in environments that have credentials.



Step 9.4: Commit + push

```

git add helix_code/tests/integration/
    auto_compaction_integration_test.go
git commit -m "$(cat <<'EOF'
test(P1-F01-T09): integration test for auto-compaction (no mocks)

```

Real Anthropic provider, conversation crafted to deliberately exceed 80% threshold of 200k window, real CompressionCoordinator with SemanticSummarization. SKIP-OK when HELIX_LLM_ANTHROPIC_KEY is unset (SKIP-OK: #P1-F01-INT). Asserts WasCompacted, post-compaction tokens less than pre, and presence of CompactionMetadata or message count reduction.

Phase: P1
Feature: F01 — Auto-Compaction
Task: P1-F01-T09

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF

)"

```
for r in github gitlab origin upstream; do git push $r main; done
```

Task 10: Challenge — runtime-evidence end-to-end

Files: - Create: helix_code/tests/e2e/challenges/auto-compaction/run.sh
- Create: helix_code/tests/e2e/challenges/auto-compaction/expected.json



Step 10.1: Write expected.json

```
{
  "must_appear_in_stdout": [
    "AUTO_COMPACTION_TRIGGERED",
    "tokens_before=[0-9]+",
    "tokens_after=[0-9]+",
    "compaction_metadata_attached"
  ],
  "must_not_appear": [
    "simulated",
    "placeholder",
    "TODO",
    "for now"
  ],
  "min_duration_ms": 1000,
  "max_duration_ms": 90000,
  "asserts_external_state": [
    {"type": "file_exists", "path": "tests/e2e/challenges/auto-compaction/.last-run-evidence.json"}
  ]
}
```

```
]
}
```



Step 10.2: Write run.sh

```
#!/usr/bin/env bash
# helix_code/tests/e2e/challenges/auto-compaction/run.sh
# Challenge: claude-code-style auto-compaction triggers at 80%
#           threshold,
# attaches metadata, and respects thrashing detection.

set -euo pipefail
cd "$(git rev-parse --show-toplevel)/HelixCode"

# Prerequisites
if [ -z "${HELIX_LLM_ANTHROPIC_KEY:-}" ]; then
    echo "SKIP-OK: #P1-F01-CHAL — HELIX_LLM_ANTHROPIC_KEY not set"
    exit 0
fi

# Build a small driver that exercises AutoCompactor end-to-end
cat > /tmp/p1-f01-driver.go <<'G0'
package main

import (
    "context"
    "encoding/json"
    "fmt"
    "os"
    "strings"
    "time"

    "dev.helix.code/internal/llm"
    "dev.helix.code/internal/llm/compression"
    "dev.helix.code/internal/llm/compressioniface"
)

func main() {
    prov, err := llm.NewAnthropicProvider(&llm.AnthropicConfig{
        APIKey: os.Getenv("HELIX_LLM_ANTHROPIC_KEY"),
        Model:  "claude-3-5-sonnet-20241022",
    })
    if err != nil { fmt.Println("provider:", err); os.Exit(1) }

    conv := &compressioniface.Conversation{ID: "chal", CreatedAt:
        time.Now()}
    for i := 0; i < 50; i++ {
        role := compressioniface.MessageRole("user")
```

```

        if i%2 == 1 { role = "assistant" }
        conv.Messages = append(conv.Messages,
            &compressioniface.Message{
                Role:    role,
                Content: strings.Repeat("Filler. ", 1000),
            })
    }

    coord := compression.NewCompressionCoordinator(prov,
        compression.WithStrategy(compression.StrategySemanticSummarization))
    ac := compression.NewAutoCompactor(prov, coord,
        compression.NewThrashingGuard(3), 0.80)
    ctx, cancel := context.WithTimeout(context.Background(),
        60*time.Second)
    defer cancel()

    result, err := ac.MaybeCompact(ctx, conv)
    if err != nil { fmt.Println("compact:", err); os.Exit(1) }

    if result.WasCompacted {
        fmt.Println("AUTO_COMPACTION_TRIGGERED")
        fmt.Printf("tokens_before=%d tokens_after=%d\n",
            result.TokensBefore, result.TokensAfter,
            result.WindowSize)
        hasMeta := false
        for _, m := range conv.Messages {
            if _, ok := compression.ReadCompactionMetadata(m); ok {
                hasMeta = true; break
            }
        }
        if hasMeta {
            { fmt.Println("compaction_metadata_attached") }
        }
    }
    out, _ := json.Marshal(map[string]interface{}{
        "was_compacted": result.WasCompacted,
        "tokens_before": result.TokensBefore,
        "tokens_after":  result.TokensAfter,
        "window_size":   result.WindowSize,
        "timestamp":     time.Now().Format(time.RFC3339),
    })
    _ = os.WriteFile("tests/e2e/challenges/auto-compaction/.last-run-evidence.json", out, 0644)
}
GO

go run /tmp/p1-f01-driver.go
echo "---"
cat tests/e2e/challenges/auto-compaction/.last-run-evidence.json

```



Step 10.3: Make executable

```
chmod +x helix_code/tests/e2e/challenges/auto-compaction/run.sh
```



Step 10.4: Run (when credentials available)

```
cd HelixCode && bash tests/e2e/challenges/auto-compaction/run.sh
```

Expected: SKIP-OK if no credentials; full run with AUTO_COMPACTION_TRIGGERED + metadata attached + evidence file written if creds set.



Step 10.5: Commit + push

```
git add helix_code/tests/e2e/challenges/auto-compaction/run.sh
      helix_code/tests/e2e/challenges/auto-compaction/
      expected.json
git commit -m "$(cat <<'EOF'
feat(P1-F01-T10): Challenge for auto-compaction with runtime
evidence
```

End-to-end Challenge that exercises AutoCompactor against real Anthropic provider with a deliberately-oversized conversation. Asserts via expected.json: must-appear "AUTO_COMPACTION_TRIGGERED" + token counts, metadata attachment, evidence-file write. SKIP-OK when no credentials.

Phase: P1
Feature: F01 – Auto-Compaction
Task: P1-F01-T10

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF

)"

```
for r in github gitlab origin upstream; do git push $r main; done
```

Task 11: Feature 1 close-out

Files: - Modify: docs/improvements/06_phase_1_evidence.md (append F01 section) - Modify: docs/improvements/PROGRESS.md (mark F01 done)



Step 11.1: Append F01 evidence section

F01 – Auto-Compaction System (claude-code)

****Timestamp:**** \$(date -Iseconds)

****Commits in order:****

1. P1-F01-T01 – Phase 1 evidence + PROGRESS init
2. P1-F01-T02 – Provider interface extension (GetContextWindow + CountTokens)
3. P1-F01-T03 – Provider implementations (per-provider methods + char-based fallback)
4. P1-F01-T04 – ThrashingGuard with 4/4 unit tests
5. P1-F01-T05 – CompactionMetadata with 3/3 unit tests
6. P1-F01-T06 – AutoCompactor with 3/3 unit tests
7. P1-F01-T07 – wired into agent.go message loop
8. P1-F01-T08 – wired ThrashingGuard.NoteUserMessage into session manager
9. P1-F01-T09 – integration test (no mocks; real Anthropic; SKIP-OK if no creds)
10. P1-F01-T10 – Challenge with expected.json (runtime evidence)

****Acceptance:****

- Unit tests: 10/10 PASS (4 ThrashingGuard + 3 CompactionMetadata + 3 AutoCompactor)
- Integration test: PASS / SKIP-OK depending on HELIX_LLM_ANTHROPIC_KEY presence
- Challenge: PASS / SKIP-OK same gating
- `go build ./...` exits 0 across the inner module
- `scripts/scan-secrets.sh` exits 0
- `make verify-foundation` exit code unchanged from Phase 0 close-out (still =2 due to documented LLMsVerifier carry-forward; F01 work did not affect it)

****Open carry-forward (F01 → Phase 3):**** per-provider native tokenizers (currently char-based fallback for all but Anthropic). Phase 3 sub-spec addresses these per-provider.



Step 11.2: Update PROGRESS.md

Mark P1-F01-T01 through P1-F01-T11 as [x]. Set Active feature pointer to F02 – Permission Rule System (next). Add Decision-log entry: 2026-05-05 – Feature 1 closed; auto-compaction landed via extension of existing internal/llm/compression infrastructure (NOT the parallel system the porting doc proposed). Per-provider real tokenizers deferred to Phase 3.



Step 11.3: Final commit + push

```
git add docs/improvements/06_phase_1_evidence.md docs/
      improvements/PROGRESS.md
git commit -m "$(cat <<'EOF'
chore(P1-F01-T11): Feature 1 (Auto-Compaction) close-out

Eleven sub-commits delivered:
- Provider interface extended (GetContextWindow + CountTokens)
- ThrashingGuard, CompactionMetadata, AutoCompactor with TDD
  discipline
- Wired into agent.go + session manager.go
- Integration test + Challenge (SKIP=OK without real credentials)

Approach: extended existing internal/llm/compression/
      infrastructure
rather than building the parallel system the porting doc
      proposed.
Per-provider native tokenizers deferred to Phase 3.

Phase: P1
Feature: F01 — Auto-Compaction
Task:    P1-F01-T11 — close-out
Evidence: docs/improvements/06_phase_1_evidence.md § F01

Co-Authored-By: Claude Opus 4.7 (1M context)
      <noreply@anthropic.com>

EOF
)"
for r in github gitlab origin upstream; do git push $r main; done
for r in github gitlab origin upstream; do printf "%-10s " "$r";
      git ls-remote --heads $r main; done
```

Plan self-review

1. Spec coverage: Every porting-doc Feature 1 item is mapped:

Porting-doc requirement	Plan task
internal/context/compaction.go	T04+T05+T06 (placed under internal/llm/compression/ instead — extends existing)
internal/context/token_counter.go	T02 (Provider interface) + T03 (implementations) — placed on existing Provider, not a parallel layer
internal/session/session_manager.go integration	T08

Porting-doc requirement	Plan task
internal/llm/provider.go interface change	T02
internal/agent/agent.go integration	T07
Auto-Compaction at 80% threshold	T06 (thresholdRatio parameter, default 0.80)
Thrashing detection (3 consecutive without user)	T04
Compaction summary metadata	T05
Unit tests	T04, T05, T06
Integration test (no mocks)	T09
Challenge (runtime evidence)	T10

2. Placeholder scan: No “TBD”, “TODO” except inside the Challenge’s expected.json must_not_appear list (intentional). No “fill in details”, no “Similar to Task N”.

3. Type consistency: - CompactionMetadata defined in T05, used in T09 + T10. Field names consistent. - AutoCompactor.MaybeCompact() signature defined in T06, called in T07 + T09 + T10. Identical. - ThrashingGuard.RecordCompaction / NoteUserMessage defined in T04, used in T06 (RecordCompaction) + T08 (NoteUserMessage). Identical. - TokenCounter interface in T06 mirrors the two new methods on Provider from T02. By design — the small interface lets AutoCompactor accept either a real Provider or a test mock.

4. Open spec-level uncertainties (carried as expected.json regex tolerance): - compressioniface.CompressionResult.TokensAfter field name verified at T06.5 — adjust if different. - compressioniface.Message.Metadata field presence verified at T05.4 — adjust if metadata lives on Conversation instead. - The integration test’s required env var HELIX_LLM_ANTHROPIC_KEY matches the key in ../helix_agent/.env (per T05’s migration). If that key isn’t set on the runner, the test SKIP-OK’s gracefully — not a defect.

Plan complete

Saved to docs/superpowers/plans/2026-05-05-p1-f01-auto-compaction.md.

Two execution options:

1. Subagent-Driven (recommended) — I dispatch a fresh subagent per task, review between tasks via superpowers:subagent-driven-development. Best for keeping each task’s context tight.

2. Inline Execution — Execute tasks in this session via superpowers:executing-plans, batched with checkpoints.

Which approach?